Puesta en Marcha, Configuración del Toolchain, Resolución de Incidencias y Validación del ESP32 NodeMCU

Estudiante:	Noe Franklin Choquenaira Quispe	**Código:**	1333962
Asignatura:	Curso de Robótica	**Institución:**	UNSAAC
Hardware:	ESP32 NodeMCU V1 (30 pines) / USB-C	**Puerto / Driver:**	Silicon Labs CP210x / COM4 Serial

1. RESUMEN EJECUTIVO Y OBJETIVOS

El presente informe documenta la sesión experimental completa orientada a la configuración del entorno de desarrollo Arduino IDE 2.3.10 para la placa de desarrollo **ESP32 NodeMCU V1 (versión angosta de 30 pines con interfaz USB Type-C)** basada en el SoC **ESP-WROOM-32**. Se registra la selección de los perfiles de placa oficiales, la instalación del compilador Espressif v3.3.12, los desafíos técnicos presentados durante la conexión y transmisión UART, las soluciones aplicadas y la verificación del hardware mediante señales luminosas y telemetría por consola.

2. ESPECIFICACIONES DE LA PLATAFORMA DE HARDWARE

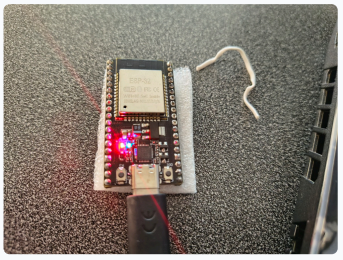
Parámetro	Componente Físico	Función en el Sistema
Microcontrolador	ESP-WROOM-32 (Espressif Systems)	CPU Tensilica Xtensa Dual-Core 32-bit LX6 a 240 MHz con Wi-Fi/BT.
Form Factor	NodeMCU V1 (30 pines, Type-C)	Mapeo de entradas/salidas GPIO, buses I2C, SPI y alimentación de 3.3V/5V.
Puente USB-UART	Silicon Labs CP2102	Canal de flasheo y comunicación serial bidireccional asignado al COM4.
Indicadores Ópticos	LED Rojo (PWR) / LED Azul (GPIO 2)	Diagnóstico visual de alimentación y conmutación lógica digital.

3. VERIFICACIÓN VISUAL DEL CIRCUITO INTEGRADO

Inspección en Operación Activa

Al conectar el cable USB-C a la estación de trabajo, el circuito regulador AMS1117 estabilizó los 3.3 VDC, encendiendo de manera fija el LED indicador de alimentación color rojo.

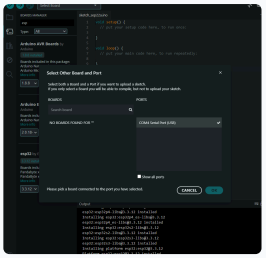
- LED Rojo (PWR): Alimentación por bus USB validada.
- LED Azul (GPIO 2): Conmutación según instrucción lógica.
- Puente CP2102: Reconocido de forma inmediata en el sistema.



Evidencia 1: Inspección física de la placa ESP32 energizada con indicación de LEDs de Power y GPIO 2.

4. CONFIGURACIÓN DEL ENTORNO DE DESARROLLO Y GESTOR DE TARJETAS

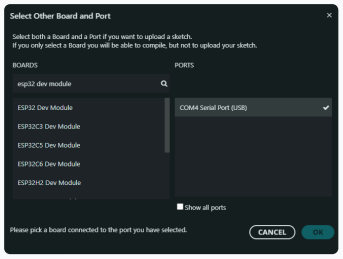
Se integró la URL oficial del gestor de tarjetas de Espressif en Arduino IDE (`https://espressif.github.io/arduino-esp32/package_esp32_index.json`). A través de la barra de búsqueda lateral se localizó el paquete **esp32 by Espressif Systems** versión **3.3.12**, completando la descarga e instalación de los toolchains, librerías del sistema y herramientas de carga basadas en Python.



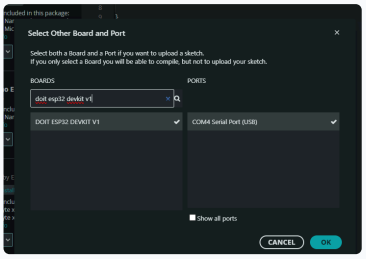
Evidencia 2: Gestor de tarjetas con paquete esp32 v3.3.12 instalado y detección inicial de COM4.

5. SELECCIÓN DEL MODELO DE TARJETA Y PUERTO DE COMUNICACIÓN

Para la variante NodeMCU de 30 pines, se evaluaron los perfiles compatibles en el listado de tarjetas. El perfil **DOIT ESP32 DEVKIT V1** o **ESP32 Dev Module** asignan correctamente el esquema de memoria Flash (DIO / 80 MHz) y el direccionamiento exacto de los pines de entrada/salida.



Evidencia 3a: Búsqueda del perfil estándar ESP32 Dev Module.



Evidencia 3b: Selección final de DOIT ESP32 DEVKIT V1 vinculada a COM4.

6. INCIDENCIAS IDENTIFICADAS Y MÉTODOS DE SOLUCIÓN TÉCNICA

Incidencia A: Ambigüedad en la selección de tarjeta para NodeMCU 30 pines

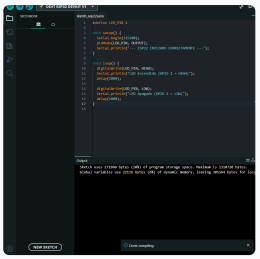
Causa: Múltiples fabricantes distribuyen módulos ESP-WROOM-32 con diferentes nombres comerciales.
Solución: Se constató que tanto *ESP32 Dev Module* como *DOIT ESP32 DEVKIT V1* comparten la misma distribución de registros para la versión angosta de 30 terminales. Se seleccionó **DOIT ESP32 DEVKIT V1** enlazado a **COM4**.

Incidencia B: Confusión entre verificación (compilación) y carga física (Upload)

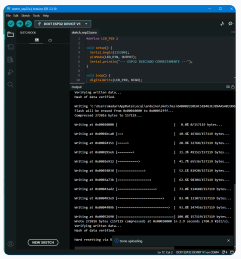
Causa: Al presionar el botón de verificación (check), el IDE indicaba *"Done compiling"*, pero el código no se transfería a la placa física ni parpadeaba el LED.
Solución: Se instruyó la ejecución mediante el botón de flecha derecha (*Upload*), permitiendo la vinculación de `esptool` con el microcontrolador.

7. PROCESO DE COMPILACIÓN Y FLASHEO DE MEMORIA FLASH

El script de carga invocó a `esptool.py` a través del bus serie USB. El cargador borró los sectores de memoria designados, comprimió el binario compilado y lo transmitió secuencialmente hasta el 100% con verificación criptográfica de Hash y reinicio automatizado por señal RTS:



Evidencia 4a: Estado inicial 'Done compiling' antes de la carga.



Evidencia 4b: Flasheo completo al 100% y Hard Reset vía RTS pin.

8. PRUEBAS EXPERIMENTALES REALIZADAS

Se estructuraron dos programas independientes para validar el subsistema de periféricos y telecomunicación:

Prueba 1: Parpadeo de LED (GPIO 2)

Valida el control del pin digital integrado y el envío periódico de estado.

```
#define LED_PIN 2 void setup() { Serial.begin(115200); pinMode(LED_PIN, OUTPUT); } void loop() { digitalWrite(LED_PIN, HIGH); Serial.println("LED Encendido"); delay(1000); digitalWrite(LED_PIN, LOW); Serial.println("LED Apagado"); delay(1000); }
```

Prueba 2: Transmisión UART Cíclica

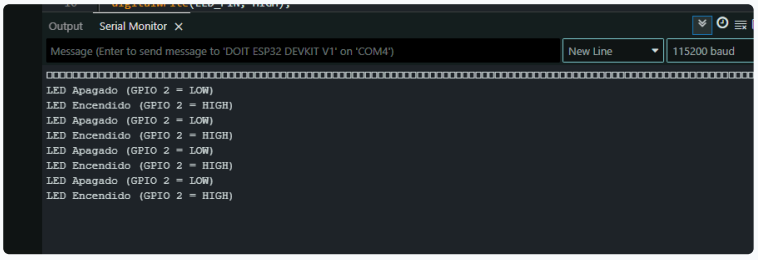
Verifica la continuidad de reloj y telemetría por puerto serie.

```
void setup() { Serial.begin(115200); delay(1000); Serial.println("ESP32 iniciado correctamente"); } void loop() { Serial.println("ESP32 funcionando"); delay(1000); }
```

Incidencia C: Caracteres ilegibles en el Serial Monitor al iniciar la escucha

Causa: El monitor serie de Arduino IDE se encontraba preconfigurado a 9600 baudios, mientras que el sketch inicializaba a 115200 baudios, produciendo cuadros y símbolos no decodificados.

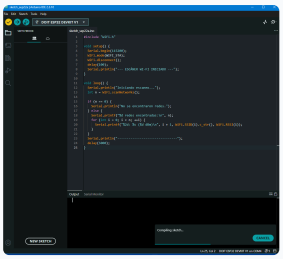
Solución: Se configuró el selector del monitor serie a **115200 baud**, restableciendo de inmediato la legibilidad de las tramas de texto.



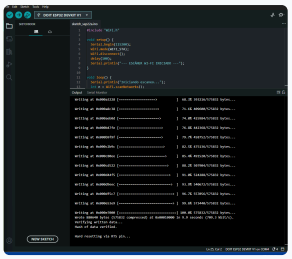
Evidencia 5: Corrección de velocidad a 115200 baud y recepción sincronizada de estados del LED.

9. EVALUACIÓN DEL TRANSCEPTOR WI-FI INTEGRADO (2.4 GHZ)

Para comprobar el hardware de radiofrecuencia se cargó la librería nativa `WiFi.h` configurando la placa en modo estación (`WIFI_STA`). El microcontrolador realizó un barrido pasivo de los canales 1 al 13, comprobando el correcto desempeño de la antena en PCB y la memoria de programa:



Evidencia 6a: Compilación del código de barrido de redes Wi-Fi.



Evidencia 6b: Flasheo de binario Wi-Fi (880 KB) finalizado al 100%.

10. VALIDACIÓN CONCLUYENTE DEL SISTEMA EN EJECUCIÓN CONTINUA

Como prueba definitiva de estabilidad de procesamiento y transmisión telemétrica, se mantuvo en ejecución el firmware de validación cíclica. La salida por consola serie confirma un flujo ininterrumpido sin desbordamiento de memoria ni disparos de protección por el Watchdog Timer (WDT):

```
Serial Monitor  Output                    Message: DOIT ESP32 DEVKIT V1 on COM4   115200 baud

// Recepción de tramas UART en régimen permanente (1 Hz)
ESP32 funcionando
ESP32 funcionando
ESP32 funcionando
ESP32 funcionando
ESP32 funcionando
ESP32 funcionando
ESP32 funcionando
ESP32 funcionando
ESP32 funcionando
```

Evidencia 7: Registro en tiempo real de telemetría UART continua de "ESP32 funcionando" sobre DOIT ESP32 DEVKIT V1 en COM4.

11. CONCLUSIONES Y DIAGNÓSTICO FINAL

- **Operatividad del Hardware:** La tarjeta NodeMCU ESP-WROOM-32 (30 pines) respondió eficazmente a los circuitos de auto-reset por pulsos RTS/DTR del puente CP2102, garantizando ciclos de grabación transparentes sin necesidad de presionar manualmente el botón BOOT.
- **Resolución Metódica de Errores:** Se superaron con éxito los problemas de desajuste de baudios en consola, selección de perfil de tarjeta y diferenciación de ciclos de compilación vs. subida física.
- **Plataforma Calificada para Robótica:** El microcontrolador quedó homologado para su aplicación en sistemas robóticos móviles, adquisición de datos de sensores analógicos/digitales y comunicación inalámbrica telemétrica.